The Dauntless Grading System

Complete Project Defense Preparation Guide

Transparent and Centralized Grading Management System
Dauntless Wolves • BSCS 2A
Updated 29 October 2026

System summary • Role features • Database and normalization
Grading formula • 100 practice questions and answers
Code walkthroughs • Demonstration plan • Testing and limitations

Use the answers to practice explaining the actual implementation. Research results, member contributions and approvals must come from your group's verified records.

Updated 29 October 2026. Reviewed implementation: main at 9c3cbbb (including merged student-profile and professor-password features). Based on the current implementation and recorded project validation. Use the sample answers in your own words. Replace research and team placeholders with your group's actual facts.

1. Opening presentation

“Our project is called The Dauntless Grading System, a Windows desktop application for managing and viewing student assessment records.

It serves three user groups: administrators, teachers, and students. Administrators manage accounts and oversee system activity. Teachers manage their classes, assessments, scores, and clarification requests. Students view their grades, monitor their performance, and request clarification about recorded scores.

The system emphasizes transparency through consistent grade calculations, visible request decisions, and audit logs that record important changes.”

2. Problem, purpose, and objectives

Problem statement

“When assessment records are managed through separate files or manual processes, students may have difficulty understanding their current standing, teachers may repeat calculations and data entry, and grade concerns may be handled without a clear record.

Our system brings those activities into one database-backed application. It provides a consistent calculation method, a formal clarification workflow, and an activity history.”

Do not claim your research proved these problems occur in every school unless your actual evidence supports that claim.

Main objectives

- Centralize student, teacher, class, assessment, and grade records.
- Calculate grades consistently across the application.
- Make score entry faster for teachers.
- Allow students to view their own academic information.
- Provide a traceable process for requesting score reviews.
- Record important actions through audit logs.
- Support CSV imports and report exports.
- Reduce invalid entries and incomplete database changes.

Roles and responsibilities

Role	Responsibilities
Administrator	Manage student and teacher accounts, activate or deactivate accounts, manage teacher employment status, use administrative class and grade-entry tools, monitor requests, and inspect audit logs.
Teacher / Professor	Access assigned classes, enroll registered students, create assessments, record scores, use the digital gradebook, review clarification requests, and view class performance.
Student	View personal grades, grade history, performance summaries, clarification requests and decisions, profile information, and change their password.

The administrator's request overview is read-only oversight. Assigned teachers handle request decisions through the review workflow.

3. End-to-end system workflow

1. An administrator registers student and teacher accounts.
2. A class offering is created and assigned to a teacher.
3. Registered, active students are enrolled in that class.
4. The teacher creates assessments with maximum scores, weights, and grading periods.
5. The teacher records scores through Record Scores or the gradebook.
6. The application validates and saves scores, calculates the student's standing, and records relevant audit entries.
7. Students view their grades and performance.
8. A student who questions a recorded score submits a clarification request.
9. The assigned teacher reviews it and may mark it Under Review, approve a correction, or close it.
10. The student sees the resulting status and available teacher response.
11. Administrators monitor requests and recorded activity.

Example scenario

“Suppose a student receives 15 out of 20 on a quiz and believes one answer was overlooked. The student submits a clarification request with a reason. The teacher reviews the assessment and, if a correction is justified, approves the request with the corrected score. The system saves the decision and score change together, recalculates the standing, and makes the decision available to the student.”

4. Features by user role

Administrator

- Dashboard totals for students, professors, classes, and open requests.
- Automatically refreshed Updates feed.
- Student and professor account directories.
- Search, filters, pagination, and account details.
- Account editing.
- Clickable Active / Inactive status badges.
- Full-time / Part-time teacher employment changes.
- Bulk activation and deactivation.
- Permanent deletion of eligible, unlinked accounts.
- Explanations when related records prevent deletion.
- Administrative class creation and quick grade entry.
- Dedicated clarification-request overview.
- Audit-log filtering and report exports.

Teacher / Professor

- Selection of an assigned class offering.
- Record Scores with student and assessment selectors.
- Enter-key score submission.
- Automatic advancement to the next student after successful submission.
- Retained focus in the score field.
- Surname-based student ordering.
- Class Grades with assessment-period filtering.

- Database-backed digital gradebook.
- Assessment creation and assessment CSV imports.
- Class-and-roster CSV imports.
- Enrollment of existing registered students.
- Request chooser and review dialog.
- Class performance summaries.
- Own-password changes from the professor dashboard, with current-password verification.
- Audit viewing and report exports.

Student

- My Grades.
- Grade History.
- Request Clarification.
- Request status and teacher-response viewing.
- My Performance.
- Course or class-offering selection.
- Read-only profile dialog with personal details, enrolled-class and open-request counts, and per-class grades and standing.
- Password changes.

Students inspect their own information. They cannot directly change their grades or approve their own requests.

5. Grading formula and interpretation

Individual assessment

Assessment Percentage = (Score / Maximum Score) x 100

Running grade

Running Grade = [sum((Score / Maximum Score) x Weight) / sum(Weights of Graded Assessments)] x 100

Only graded assessments with valid positive maxima and weights contribute.

Assessment	Score	Percentage	Relative weight
Quiz	16/20	80%	20
Activity	45/50	90%	30
Exam	80/100	80%	50

Example: [(0.80 x 20) + (0.90 x 30) + (0.80 x 50)] / (20 + 30 + 50) x 100 = 83%.

The current passing threshold is 75%.

- Blank: not graded yet; excluded from the running calculation.
- Zero: an actual recorded score; included in the calculation.
- No graded assessments: no running grade yet; display an appropriate pending or Not Yet state.
- Relative weights: normalized over assessments that currently have scores.
- Category averages: summaries for display, not additional contributions added on top of the assessments.

The running grade can change when more work is graded. It is not automatically a completed final grade.

The Class Grades Midterm / Finals filter selects assessment columns. Its displayed RunningGrade remains the overall running grade. Do not describe that filter as a separate term-grade calculation.

6. Technical structure

“The system uses VB.NET and Windows Forms for the desktop interface, .NET 8 as its application framework, and a MySQL database for persistent storage. The interface calls service classes that handle operations such as authentication, grading, enrollment, imports, and request reviews.”

Part	Purpose
Forms / interface	Collect user input and display information.
Service classes	Validate operations and coordinate application rules.
GradeCalculator	Apply the shared grading formula.
DatabaseHelper	Create database connections and execute database commands.
MySQL database	Persist accounts, classes, assessments, grades, requests, and audit records.
AppSession	Hold information about the currently logged-in user.
Export services	Prepare supported reports and CSV outputs.

This is a layered desktop application with direct database connectivity. Do not describe it as a web application, microservice system, or API-based architecture.

Main database tables

Table	Stored information
admin	Administrator accounts.
teacher	Teacher accounts and employment information.
student	Student accounts and academic profile information.
course	Course definitions.
class	Specific course offerings and assigned teachers.
enrollment	Student membership in a class.
assessment	Assessment details, maxima, weights, and periods.
studentgrade	Assessment scores associated with enrollments.
revisionrequest	Clarification requests and review status.
auditlog	Recorded actions and change details.

“A course can have multiple class offerings. A class has enrollments and assessments. Each student's assessment score is connected through their enrollment. A clarification request refers to a specific grade record.”

The course is the subject definition. A class is a particular offering identified by course code, section, semester, and academic year.

7. Improvements made during development

Area	Improvement
Grade consistency	Replaced conflicting worksheet calculations with a shared weighted calculation based on saved assessment scores.
Saved gradebook data	Connected editable assessment cells to persistent database records.
Gradebook editing reliability	Addressed a reload-during-edit problem that could cause a DataGridView reentrancy exception.

Area	Improvement
Fast score entry	Added Enter submission, success-only student advancement, and retained input focus.
Student ordering	Applied surname-based ordering and improved surname-first gradebook display.
Class selection	Made displayed class data refresh when the selected offering changes.
Assessment access	Made Add Assessment open its own dialog without unnecessarily opening the gradebook.
Request reviews	Replaced the basic Yes / No prompt with a chooser and structured review dialog.
Student feedback	Made request decisions and available teacher responses visible to students.
Admin requests	Added dates, details, filters, automatic refresh, and exports.
Admin account management	Added status controls, employment changes, bulk actions, details, and clearer deletion feedback.
Admin startup reliability	Fixed initialization assumptions that caused null-reference crashes.
Imports	Added validated workflows for class rosters, enrollment, and assessments.
Atomic operations	Strengthened transactions so related records save or roll back together.
Reports	Improved pagination, wide-column handling, and long-text output.
Layout	Fixed clipped labels, analytics cards, progress labels, and toolbar controls.
Window behavior	Connected the gradebook lifecycle to its professor window.
Naming and design	Replaced captions such as Form1 / frmprof with clearer titles and made dialogs more consistent.
Documentation	Prepared the user manual and validated CSV testing files.

Describe these as iterative improvements based on testing and feedback. Do not claim a formal research methodology unless it matches your actual project documentation.

8. Likely panel questions and sample answers

Purpose and scope

1. What is your system about?

It is a desktop grading application that connects administrators, teachers, and students through shared academic records, consistent grade calculations, and a documented clarification process.

2. Why did you develop it?

We wanted to make grade recording, viewing, and clarification more organized. The system also provides a history of important actions so users can understand changes.

3. What is the main contribution of your project?

Its main contribution is integrating grade management with transparency: students can view their records, request reviews, and see decisions, while teachers and administrators have traceable workflows.

4. What makes it different from an ordinary spreadsheet?

It adds user roles, persistent relationships between records, validation, a clarification workflow, and audit history. A spreadsheet remains flexible, but those controls usually need to be managed separately.

5. Is it a complete learning management system?

No. Its scope centers on grades, assessments, enrollment, performance summaries, and clarification requests. It does not currently provide a complete online teaching or assignment-submission platform.

6. Who are the intended users?

Administrators, teachers, and students. Each role has a different set of functions and access.

Technology and deployment

7. Why use a desktop application?

A desktop application fits the current Windows-based implementation and provides a familiar interface for tabular grade entry. A web version would be a separate future development.

8. Why VB.NET and Windows Forms?

They support event-driven desktop development and familiar controls such as forms and data grids. They also fit the development environment and skills used for this project.

9. Why MySQL?

Our data has clear relationships. MySQL supports relational tables, constraints, queries, and transactions that help preserve those relationships.

10. Is the system cloud-based?

The application itself is a Windows desktop system. The database location depends on configuration. Using a cloud development environment does not make the application a cloud-native system.

11. Does it need the internet?

It needs access to its configured database. A local database does not require public internet; a remote database requires an appropriate network connection.

12. Can multiple computers use it?

The architecture allows clients to connect to the same configured database when networking and permissions are properly set up. We should distinguish that capability from large-scale deployment testing.

Authentication, access, and account controls

13. How are users authenticated?

The application looks up the account, verifies the password hash, checks the relevant account status, and opens the interface for the authenticated role.

14. Are passwords stored in plain text?

New passwords in the active authentication workflow are stored using salted PBKDF2-SHA256 hashes. Existing supported SHA-256 hashes can be upgraded after a successful login.

15. What is a salt?

A salt is a random value included in password hashing. It means two users with the same password do not need to have the same stored hash.

16. Does hashing mean the password can be decrypted?

No. Hashing is one-way. The application verifies a password by deriving a value from the entered password and comparing it with the stored value.

17. What prevents SQL injection?

Database operations use parameterized values so user input is treated as data rather than SQL commands. We also validate expected input.

18. Can a student view another student's grades?

The student workflows query records belonging to the logged-in student. Ownership checks are important because hiding a button alone would not protect data.

19. Can a teacher modify any class?

Protected teacher operations validate that the class or assessment belongs to that teacher.

20. What happens when an account is deactivated?

Inactive student and teacher accounts cannot log in through the normal workflow. Protected write operations also recheck account availability so an already-open session cannot continue those writes.

21. What is the difference between deactivation and deletion?

Deactivation preserves the account and related history while restricting access. Deletion permanently removes an eligible account record.

22. Why can some accounts not be deleted?

They have related records such as enrollments, classes, requests, or audit activity. Blocking deletion protects those relationships and preserves history. Deactivation is the appropriate alternative.

23. Does an administrator approve clarification requests?

The current admin request overview supports monitoring. Teachers assigned to the relevant classes make review decisions.

Grade calculations

24. How does the system calculate grades?

It calculates each assessment's score-to-maximum ratio, applies its relative weight, and normalizes the result over graded assessments.

25. Why use weights?

Weights let assessments contribute according to their intended importance instead of assuming all assessments contribute equally.

26. Must weights total exactly 100?

The calculator uses relative weights and normalizes them. A total of 100 is convenient to explain, but the formula does not depend on that exact total.

27. Why exclude ungraded assessments?

An ungraded assessment does not yet represent a measured result. Treating it as zero would incorrectly lower the current running grade.

28. What happens if the student receives zero?

Zero is saved as a real score and contributes to the running grade.

29. What if no assessments have been graded?

The system has no running grade to calculate yet. It should show a pending or Not Yet state rather than a failing zero.

30. What is the passing grade?

The current threshold is 75 percent. The application centralizes that threshold in the grade calculator.

31. Is this a transmuted grade?

The current shared calculation uses weighted raw-score percentages. We should not describe it as a transmutation system.

32. What happens when a score exceeds the maximum?

The input is rejected, and the saved grade should remain unchanged.

33. What score precision is supported?

Relevant score, maximum, weight, and requested-point inputs support up to two decimal places. Excess precision is rejected instead of silently accepted.

34. Does the performance chart determine the grade?

No. It summarizes the underlying data. The grade calculation comes from the shared calculator, not the visual bar.

35. Why is the same calculator used across screens?

It prevents the gradebook, class sheet, and student views from applying different grading rules.

Score entry and names

36. What makes score entry faster?

After a successful submission, the student selector advances to the next student and the score field remains focused. Teachers can continue typing without repeatedly using the mouse.

37. What happens after an unsuccessful submission?

The system keeps the current student and entered value so the teacher can correct the problem. It does not advance as if the save succeeded.

38. What happens after the last student?

The workflow indicates completion and stays on the last student rather than automatically wrapping to the beginning.

39. How are names sorted?

Students are ordered by surname, with additional tie-breakers. Because the database stores a full name in one field, ambiguous names are best stored in an explicit surname-first format.

40. Does the system know every student's middle initial?

No. There is no separate middle-name field in the current schema. The display should preserve available name information rather than invent initials.

Clarification requests

41. What is a clarification request?

It is a formal request asking the teacher to review a particular recorded assessment score.

42. What are the request statuses?

Pending, Under Review, Approved, and Closed.

43. What does Under Review mean?

The teacher is considering the concern, but it has not reached a final decision.

44. What does Closed mean?

The request is finished without applying an approved correction through that request. The reason or response provides context.

45. Does a request automatically change a grade?

No. The student's explanation or requested adjustment does not directly update the score. The teacher must review and approve an appropriate correction.

46. Can the student type '+5' in the reason and receive five points?

No. Free-text reasons do not execute grade changes. The approved score comes from the review operation.

47. How does approval apply a corrected score?

A teacher can supply a corrected score. Where a valid structured point request is used, the service can apply that adjustment within the assessment maximum. A plain re-check requires an explicit corrected score.

48. Can the same request be approved twice?

The review service checks whether the request is still open. Approved and Closed requests cannot go through another normal approval.

49. Can a student submit duplicate open requests for the same grade?

The service prevents another open request for the same grade while one is Pending or Under Review.

50. Can students see why their request was closed?

Yes. The student request view displays the available review response and review information. Older records with missing information use a fallback rather than inventing a reason.

51. What if the teacher approves but the score save fails?

The related changes are transactional. The approval and score update should roll back together rather than leave a misleading Approved status.

Integrity, audit history, and refresh

52. What is a database transaction?

It groups related database changes into one operation. Either all required changes commit, or they are rolled back.

53. Where are transactions especially important in your system?

Score batches, request approval, imports, bulk account changes, and password changes that must remain consistent with their audit records.

54. How do you handle two operations affecting the same record?

Important workflows use transactions, database constraints, and row locks where implemented. Request updates also verify that the request is still in an open state.

55. What is an audit log?

It records important actions, including who performed them, when they occurred, and available information about the affected record and change.

56. How is audit history different from grade history?

Audit history covers broader application activity. A student's grade history focuses on changes belonging to that student's grade records.

57. Are audit logs impossible to change?

No. They support application accountability, but a person with sufficient database privileges could alter database records. We do not claim cryptographically tamper-proof logging.

58. How do automatic updates work?

Relevant forms periodically query the database. For example, the admin dashboard checks for updates roughly every two seconds, while the admin request overview checks roughly every five seconds, plus query time.

59. Is that real-time push technology?

No. It is periodic polling. It reduces manual refreshes, but there can be a short delay.

60. What happens when the database is unavailable?

The operation should report failure rather than pretend it saved. Relevant read screens preserve displayed results and provide an error or retry path.

61. Does the system work offline and synchronize later?

No. It currently requires a database connection for its database-backed operations. Offline synchronization is outside the current scope.

Imports and exports

62. Why do you use CSV imports?

They reduce repetitive entry when creating assessments, importing class rosters, or enrolling registered students.

63. What CSV formats are supported?

There are separate formats for class rosters with assessment columns, enrollment-only files, and assessment-definition files.

64. Can importing a class create student accounts?

No. Students must already be registered and active. The import validates their IDs and names.

65. Why validate both student ID and name?

The ID identifies the account, while checking the name helps catch accidental mismatches before records are associated with the wrong student.

66. What happens to blank CSV scores?

They remain ungraded. A written zero is treated as a recorded zero.

67. What if a later CSV row is invalid?

The supported transactional import should reject or roll back the operation without leaving only part of the file saved.

68. Are all repeated imports safe from duplication?

No. Enrollment imports preserve existing enrollments, but importing an assessment-definition file again creates another assessment set with available titles. The user should not repeatedly import a successful assessment file expecting an update.

69. What prevents duplicate enrollments?

The database has a unique relationship for each student-and-class combination, supported by application checks.

70. What prevents duplicate class offerings?

The database enforces uniqueness for the combination of course code, section, semester, and academic year.

71. Why offer PDF and CSV exports?

PDF supports readable reports for viewing or printing. CSV supports structured data exchange and further inspection in compatible tools.

72. Can you export large tables?

The exporter supports pagination and groups wide column sets across pages while repeating student identification where needed. Final printer behavior still depends on Windows and the configured PDF printer.

Window behavior and testing

73. What happens if the professor closes the main window while the gradebook is open?

The gradebook is connected to the professor window's lifecycle so it closes with it rather than leaving an unintended extra window open.

74. How did you test the system?

We used backend integration checks against a disposable database, targeted regression checks, invalid-input cases, and Windows interaction test projects and manual-check instructions.

75. How many tests passed?

The recorded backend suite passed 211 checks. A separate CSV-pack validation run passed 48 checks covering valid files and intentional rejection cases. A further targeted run passed 21 checks for student-profile isolation, summaries and teacher password changes. These are separate runs and are not evidence of exhaustive coverage.

76. Does that mean the whole application is guaranteed bug-free?

No. Those checks provide evidence for the tested cases. Windows interface behavior, scaling, and printing require Windows testing, and untested situations may still reveal issues.

77. What kinds of failures did you test?

Out-of-range scores, excess decimal places, unauthorized operations, duplicate decisions, invalid imports, account deactivation, and deliberately induced database failures to verify rollback.

78. Why use a disposable database?

The tests intentionally create and modify records. A disposable database lets us test those operations without damaging actual academic records.

79. What were your most significant technical challenges?

Keeping calculations consistent across screens, saving gradebook edits without disrupting active editing, ensuring partial operations roll back, and refreshing data without resetting the user's work.

80. How did user feedback influence the system?

Feedback revealed clipped controls, confusing review dialogs, unnecessary window openings, slow score-entry steps, and missing refresh behavior. We used that feedback to improve both usability and reliability.

9. Limitations and honest answers

Possible question	Defensible answer
Is it fully production-ready?	It is a working academic project. Broader deployment would require stronger operational security, backup procedures, deployment testing, and performance evaluation.

Possible question	Defensible answer
Can it handle thousands of concurrent users?	We have not established that through large-scale load testing.
Does it have automatic backups?	We should not claim a built-in automated backup feature. Database backup and restoration need an operational procedure.
Does it send email or SMS notifications?	That is not part of the current implemented workflow.
Can users access it through a phone browser?	No. The current application targets Windows desktop.
Does it detect cheating or predict failure using AI?	No. Performance summaries are calculated from recorded scores, not predictive AI.
Can it prove a teacher graded fairly?	It makes recorded scores, changes, and review decisions more traceable. It cannot independently judge the fairness of the original assessment.
Can someone change the database outside the application?	Someone with sufficient database privileges can. Database access controls remain important.
Is account security complete?	Password hashing and access checks are present, but stronger password policy, login throttling, centralized deployment controls, and further security testing are worthwhile improvements.
Is it automatically compliant with privacy law?	No software feature alone establishes legal compliance. Deployment requires appropriate policies, permissions, retention practices, and institutional review.

Suitable future improvements

- Separate surname, given-name, and middle-name fields.
- Stronger password requirements and login-attempt controls.
- A central API instead of distributing direct database access to clients.
- Automated backup and verified restoration procedures.
- Notifications for request decisions.
- A web or mobile interface.
- More configurable institutional grading policies.
- Formal load testing and larger usability evaluations.
- Better accessibility and broader display-scaling coverage.
- A clearer policy for historical requests if a teacher assignment changes.

Present these as future work, not existing features.

10. Research, teamwork, and development questions

What methodology did you use?

“We used [the methodology documented in our paper]. During implementation, we developed and refined features through repeated testing and feedback.”

Do not say Agile, Scrum, Waterfall, or RAD merely because the panel expects a methodology. Use the one your group actually followed and documented.

How many respondents evaluated it?

“Our evaluation involved [actual number] respondents using [actual instrument].”

If no formal evaluation has been completed: “We have conducted functional testing and demonstrations, but we have not yet completed a formal respondent-based evaluation.”

What percentage improvement did you achieve?

“We have not measured a defensible percentage improvement. The implemented workflow reduces repeated interaction—for example, score submission advances to the next student—but timing studies would be needed to quantify that.”

What did each member contribute?

Prepare a truthful list before the mock defense. Relevant areas include requirements and documentation, database design and setup, interface development, service and calculation logic, testing and bug reporting, and presentation and demonstration preparation. Only assign work people actually performed.

Did you use AI?

Answer honestly and follow your school's disclosure requirements:

“We used AI assistance for parts of troubleshooting, code refinement, testing support, and documentation. We are responsible for reviewing the output and understanding the behavior of the submitted system.”

Do not claim that all code was independently written without assistance.

Can you explain a piece of your code?

- GradeCalculator: how percentages and running grades are computed.
- AuthService: how login and password changes work.
- GradeService: how scores and assessments are validated and saved.
- RevisionRequestService: how ownership, status changes, and approval work.
- DatabaseHelper: how the application connects to the database.

You do not need to memorize every line. Understand the inputs, rules, database effects, and failure behavior.

11. Recommended live demonstration

12. Introduce the roles and workflow: who manages accounts, records grades, and views them.
13. Log in as an administrator. Show account search, details, status controls, dashboard updates, and request oversight.
14. Log in as a teacher. Select an offering and explain why section, semester, and year matter.
15. Record a score using Enter. Show automatic student advancement and retained input focus.
16. Show the gradebook. Explain raw scores, percentages, and the running grade.
17. Demonstrate one validation case: try a score above the maximum and show rejection.
18. Log in as the student. Show the saved grade and submit a clear clarification request.
19. Review it as the teacher. Show the reason, corrected score, and response before approving.
20. Return to the student view. Show the decision and available response.
21. Show the audit trail. Connect the recorded activity to the action just demonstrated.
22. Demonstrate an import or export if time allows. Use prepared test CSVs and a separate test class.

Use test records and agree on demonstration accounts beforehand. For deletion, use an unlinked test account; a linked account should correctly demonstrate the protection message.

12. Claims to avoid

- “It is 100% secure.”
- “It has no bugs.”
- “All tests were executed on Windows.”
- “Every account can be permanently deleted.”
- “Blank grades are zero.”

- “Typing requested points automatically changes the grade.”
- “The administrator approves every request.”
- “The application works offline.”
- “The audit trail is impossible to alter.”
- “The system uses AI to predict performance.”
- “We proved it is faster by a specific percentage,” without measurements.
- “It is cloud-based,” just because development happened in a cloud environment.

When you do not know an answer

“That is outside what we have verified so far. Based on the current implementation, [state what you know]. We would need [a specific test or improvement] before making a stronger claim.”

13. Closing presentation

“The Dauntless Grading System brings grade recording, student viewing, clarification requests, and administrative oversight into one application.

During development, we improved calculation consistency, database reliability, score-entry efficiency, request transparency, and interface usability. The system's value is not only that it stores grades, but that it gives users a clearer and more traceable process for managing them.

We also recognize its current limitations and have identified security, deployment, and evaluation improvements for future development.”

14. Last-minute review card

- Three roles: Admin, Teacher, Student.
- Platform: VB.NET, .NET 8, Windows Forms, MySQL.
- Purpose: organized grades, transparency, and traceable reviews.
- Formula: weighted score ratios normalized over graded assessments.
- Passing threshold: 75%.
- Blank is ungraded; zero is a real grade.
- Requests: Pending, Under Review, Approved, Closed.
- Teachers decide; admin request overview monitors.
- Transactions prevent partial saves.
- Audit logs support accountability, not tamper-proof guarantees.
- Automatic updates use polling, not push notifications.
- Recorded validation: 211 backend checks; separate CSV pack validation: 48 checks; targeted profile/password suite: 21 checks.
- Windows interaction projects compile; runtime, scaling, and final printing checks require Windows testing.
- Explain actual contributions and research evidence honestly.

15. Official documentation title and current implementation

Documentation title: Transparent and Centralized Grading Management System.

Application name: The Dauntless Grading System. Explain that these names refer to the same project, not two separate products.

Group: Dauntless Wolves, BSCS 2A. The supplied concept paper lists Hope Gian Condes, Carl Jade Lalata, Geoff Andrei Mariano, Charles Oropilla and Ron Jacob Puyat. Confirm spelling and actual responsibilities with each member. The documentation identifies Mrs. Rebecca R. Fajardo as professor.

The concept paper dated August 23, 2026 identified fragmented spreadsheet records, delayed visibility, overwritten information, limited traceability and manual interpretation of grades. The implemented system responds through a shared database, current running grades, scoped history, performance summaries and formal clarification requests. These are project motivations, not a claim that a statistical study proved their prevalence.

A two-minute opening script

“Good day. We are the Dauntless Wolves, presenting our Transparent and Centralized Grading Management System, shown in the application as The Dauntless Grading System.

Our concept paper identified concerns with separate spreadsheet records: students may receive delayed feedback, grade changes may overwrite earlier values, and teachers may spend time consolidating and interpreting records. Our objective was to organize these activities in a database-backed application with clearer access and accountability.

The system has three roles. Administrators manage accounts, class setup and oversight. Teachers manage assigned classes, assessments, scores and clarification decisions. Students view their own grades, history and performance and submit concerns about recorded scores.

The application uses VB.NET, .NET 8 Windows Forms and MySQL/MariaDB. A shared calculator uses relative assessment weights over graded work. Database transactions keep related score, request and audit operations consistent.

Important features include a saved digital gradebook, faster Enter-based score entry, validated CSV imports, report exports, request responses and audit history. Our latest additions provide a read-only student profile summary and professor password changes using existing data and services.

We will demonstrate a score entry, student viewing, clarification review and the resulting audit trail. We will also explain the database relationships, testing evidence and the limitations of the current desktop implementation.”

16. Database and normalization defense

Relationship map to memorize

- Course → Class: one course can have many offerings; each class references one course.
- Teacher → Class: one teacher may handle many classes; each class has one assigned teacher.
- Student ↔ Class: many-to-many, resolved by Enrollment.
- Class → Assessment: one class has many assessments; each assessment belongs to one class.
- Enrollment ↔ Assessment: grade records are held in StudentGrade, unique per enrollment-assessment pair. The application checks that both belong to the same class.
- StudentGrade → RevisionRequest: a grade may have multiple historical requests, but at most one is open at a time.
- Teacher → RevisionRequest: the final reviewer is optional before approval/closure; Under Review does not populate the final review fields.
- AuditLog → account actors: exactly one student, teacher or admin actor per entry. Its target entity fields are textual identifiers rather than foreign keys.

Normalization explanation

UNF: a hypothetical spreadsheet may contain repeated quiz columns, lists of classes, and repeated student/course/teacher details. This is a design illustration; do not present it as a separate deployed table.

1NF: represent atomic fields and one grade entry per enrollment-assessment combination. Repeating request and audit records belong in separate relations.

2NF: with the illustrative composite key (enrollment_id, assessment_id), enrollment details depend only on enrollment_id and assessment definitions only on assessment_id. Separate them, retaining the pair-dependent score in StudentGrade.

3NF: remove transitive descriptive dependencies. Student details belong to Student; class offering details to Class; course_name to Course; teacher details to Teacher. Separate account, request and audit entities by their own keys.

Enrollment.running_grade is intentionally cached derived data. Acknowledge this controlled redundancy rather than claiming that every stored value is independent. Score-saving operations maintain the cache, while relevant displays recompute using the common calculator.

Constraints and validation

Rule	Enforcement
Unique account ID	Primary key in each role table.
Unique email	Unique within each role table; not globally across roles.
One enrollment per student/class	UNIQUE(student_id, class_id).
One grade per enrollment/assessment	UNIQUE(enrollment_id, assessment_id).
Unique class offering	UNIQUE(course_code, section, semester, academic_year).
Positive assessment maximum	Database CHECK and application validation.
Weight	Database permits 0–100; creation workflow requires greater than zero, at most 100.
Score range and matching class	Application validation across enrollment and assessment records.
One open request	Application workflow with transactional checks and locking.
One audit actor	Database CHECK for exactly one non-null actor ID.
Password hashing	Application logic; NOT NULL does not prove a value is hashed.

Deletion rules differ by relationship. Protected references block deletion, grades cascade from a deleted enrollment/assessment, requests cascade from their deleted grade, and the reviewer reference has SET NULL behavior. The admin account-deletion service applies additional linked-record protection. Do not infer a user-facing grade-delete feature from database cascades.

17. Additional panel questions: current features and design tradeoffs

81. Why are there ten tables rather than one large table?

The tables represent different entities and relationships. Separating them avoids repeatedly copying course and teacher details into every student score, and supports keys, constraints and controlled access.

82. Why is Enrollment necessary?

It resolves the many-to-many relationship between students and classes and provides the context for each student's class grades.

83. Why does StudentGrade reference Enrollment instead of only Student?

The enrollment identifies the student's exact class participation. The application also checks that the assessment belongs to that same class.

84. Why have both grade_id and a unique enrollment-assessment pair?

The grade ID gives revision requests and audit targets a convenient stable identifier. The unique pair prevents duplicate grade records for the same assessment and enrollment.

85. Are primary keys and foreign keys the same?

A primary key uniquely identifies a row. A foreign key connects a row to an existing related row and helps prevent broken references.

86. Is an email unique throughout the whole system?

No. Uniqueness is enforced within each account table. The selected role helps resolve authentication when role records overlap.

87. Why are some checks in the backend rather than SQL CHECK constraints?

Some rules require related tables or current actor information, such as assessment ownership, matching classes and permitted request transitions. Row-level constraints alone do not enforce those complete workflows.

88. Does marking Under Review record the reviewer and review time?

It records an audit action. The final reviewed_by and reviewed_at fields are populated when the request is Approved or Closed.

89. Does inactive student status stop teachers from grading them?

No. It restricts student login and new enrollment. Assigned teachers may still maintain existing scores and review existing requests. This preserves access to historical academic records.

90. What is in the new student profile?

It displays existing personal details, enrolled-class count, open-request count and each class's current running grade and standing. It is read-only and uses the logged-in student's identity.

91. Did that profile require another table?

No. It reads existing student, enrollment, class, course, assessment, grade and request records. The summary is not a new stored academic record.

92. What counts as an open request?

Pending and Under Review. Approved and Closed are excluded.

93. Does the profile calculate a new overall average across courses?

No. It shows grades per class. An overall cross-course average would require an agreed policy, such as whether to weight by course units.

94. How can a professor change a password?

Use Change Password beside Log out, enter the current password, enter and confirm a new password, then submit. The existing service checks the current password and active account, saves a salted hash and audit entry transactionally.

95. What is the current minimum password length?

Four characters in the current implementation. This is a project limitation; a stronger password policy is recommended before broader deployment.

96. Is the audit log the same as a backup?

No. It records actions and selected change information. It is not a complete database backup, and there is no general automatic undo or restore feature in the application.

97. Can every proposed concept-paper feature be claimed as completed?

No. Explain the implemented scope. Home access depends on deployment and networking; historical predictive trends, push notifications and automatic recovery should not be claimed merely because they were proposed.

98. Does the Active Classes tile prove that classes have active/inactive status?

No. The current schema has no class-status field; the dashboard counts class records. Explain the actual data rather than inferring a policy from the caption.

99. What does the new testing evidence prove?

The 21 targeted backend checks support tested profile isolation, counts, weighted summaries and teacher password behavior. They do not execute Windows Forms layouts, keyboard interaction or printer behavior.

100. What would you improve first before actual institutional deployment?

Strengthen password and database access policies, establish verified backups and restoration, complete Windows interaction and deployment testing, and measure performance under the expected workload. Priorities should reflect the institution's requirements.

18. Code walkthrough preparation

Explain one score save as a sequence: capture the selected enrollment and assessment; verify the actor and ownership; confirm matching class, numeric precision and range; read/lock the existing score; write the grade and assessment remarks; refresh the cached running grade; write the audit entry; commit. On a required failure, roll back and keep the user on the same entry.

Explain one request approval: verify an active assigned teacher; load/lock the open request; determine a valid corrected score; update the decision and final review fields; use the shared score-saving path; add audit information; commit together. A plain re-check requires a corrected score, and text in the student's reason does not execute arithmetic.

Explain the profile: require a student session; read only that identity's details and enrollments; load related saved grades and open-request count; calculate per-class standing using the common calculator; display without editing records.

Explain the password dialog: capture the current teacher session; validate required fields, confirmation and a different new password; call the existing service; verify old password; write the hash and audit together. Do not reveal password contents or hashes during a demo.

Useful code locations: Services/GradeCalculator.vb; Services/GradeService.vb; Services/RevisionRequestService.vb; Services/AuthService.vb; Services/StudentProfileService.vb; Services/AccountAccess.vb; Core/DatabaseHelper.vb; Core/SecurityHelper.vb.

19. Demonstration timing and recovery plan

Approximate time	Demonstration
0–2 minutes	Purpose, users, objectives and scope.
2–4 minutes	Database relationship explanation and administrator overview.
4–7 minutes	Teacher selection, Enter-based score entry, gradebook and

Approximate time	Demonstration
	invalid score.
7–9 minutes	Student grades/profile and clarification submission.
9–11 minutes	Teacher decision, student response and audit evidence.
11–12 minutes	Reports, testing, limits and conclusion.

Agree on a demo class, student, teacher and assessment. Use synthetic test records and verify credentials beforehand. Prepare a known-valid CSV and a known-invalid one. Check the database service, dbconfig.json, printer and display scaling before the presentation. Keep an untouched database backup and know the restore procedure; do not restore over shared active records during a demo.

If networking or printing fails, state the observed failure honestly, show the relevant error handling, and use labeled screenshots as supporting evidence. Do not claim a screenshot is a live successful operation. Do not repeatedly edit main or merge untested code during the defense.

Change a test professor password only if the team agrees and the replacement is known to the account holder. It is acceptable to demonstrate validation/cancellation instead of changing the shared demo credentials live.

20. Documentation and team handoff checklist

- Use consistent project naming and table names across slides, ERD, dictionary and source code.
- Include teacher.employment_type in the final dictionary and ERD where attributes are shown.
- Explain relative weights, zero versus missing scores, optional Under Review and protected deletion correctly.
- Submit the actual SQL script, Visual Studio solution, documentation and relevant test evidence.
- Confirm that dbconfig.json matches the submitted database name; do not expose deployment credentials in public slides or files.
- Update screenshots for the new profile and professor-password dialogs after Windows verification.
- Record actual Windows test outcomes, dates and testers; do not convert pending checks into Passed.
- Verify team contributions, methodology, respondents and approvals against actual records.
- Follow the instructor's AI disclosure and reference requirements.
- Give each member a topic to explain: database relationships, grade math, authorization, transactions, interface flow or tests. Responsibilities must reflect actual work.

Five facts every presenter should know

23. Running grade normalizes relative weights over graded assessments, and the threshold is 75%.
24. Students see their own records; assigned teachers decide requests; admins have oversight and authorized grade entry.
25. Transactions prevent supported multi-record operations from committing only partially.
26. Audit history records actions but is neither a complete backup nor tamper-proof against privileged database access.
27. The project is a Windows desktop application; polling, not push technology, refreshes relevant screens.